SOLVED ASSIGNMENT

Kubernetes & Minikube on AWS EC2

Aim: Provision an AWS EC2 instance, install Docker, kubectl and Minikube, start a Kubernetes cluster, deploy Nginx, expose it using NodePort/port-forwarding, verify browser access, and clean up AWS resources.

The solution follows the uploaded 4-page AWS EC2 / Minikube lab guide and preserves its specified instance type, ports, commands, Kubernetes manifests, and workflow.

1. TARGET ARCHITECTURE

```
AWS EC2 (Ubuntu 24.04/22.04 LTS)
        ↓
Docker Engine
        ↓
Minikube Cluster (Docker driver)
        ↓
Nginx Deployment (2 replicas)
        ↓
NodePort Service : 30080
        ↓
EC2 Public IP : 30080
        ↓
Web Browser
```

Target machine: t3.medium, 2 vCPU, about 4 GB RAM, and 25–30 GB gp3 storage. The guide specifies NodePort 30080 for browser access.

2. SECURITY GROUP RULES

Type	Protocol	Port	Source	Purpose
SSH	TCP	22	Your IP	EC2 terminal access
Custom TCP	TCP	30080	0.0.0.0/0	Nginx NodePort browser access
HTTP (optional)	TCP	80	0.0.0.0/0	Optional / proxy

3. PHASE 1 – AWS EC2 INSTANCE PROVISIONING

Step 1.1: Launch the EC2 Instance

In AWS Console, go to **EC2** → **Launch Instances** and use:

Setting	Value
Name	k8s-minikube-lab
AMI	Ubuntu Server 24.04 LTS (HVM), SSD Volume Type, x86_64
Instance Type	t3.medium
Key Pair	k8s-key.pem (example)
Storage	25 GB gp3

The guide notes that Minikube needs at least 2 vCPUs and 2–4 GB memory; the specified t3.medium is used for the lab.

Step 1.2: Configure the Security Group

Add the inbound rules shown above, especially TCP 30080 from 0.0.0.0/0 for browser access.

Step 1.3: Connect by SSH

```
chmod 400 k8s-key.pem
ssh -i "k8s-key.pem" ubuntu@<EC2-PUBLIC-IP>
```

4. PHASE 2 – DOCKER, KUBECTL & MINIKUBE INSTALLATION

Step 2.1: Update System and Install Docker Engine

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl wget apt-transport-https ca-certificates gnupg lsb-release

sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu

sudo apt update && sudo apt install -y docker-ce docker-ce-cli containerd.io

sudo usermod -aG docker $USER
newgrp docker
docker ps
```

Step 2.2: Install kubectl

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
kubectl version --client
```

Step 2.3: Install Minikube

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
rm minikube-linux-amd64
minikube version
```

5. PHASE 3 – START AND VERIFY THE CLUSTER

Step 3.1: Start Minikube using Docker driver

```
minikube start --driver=docker --cpus=2 --memory=3072
```

AWS note: Minikube runs inside a Docker container on EC2. Its internal IP is isolated from the laptop/public internet, so the application must be exposed through the EC2 host.

Step 3.2: Verify cluster health

```
minikube status
kubectl get nodes
kubectl cluster-info
```

6. PHASE 4 – DEPLOY NGINX APPLICATION

Step 4.1: Create and Apply the Deployment Manifest

```
cat <<EOF > nginx-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
  labels:
    app: web-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
      - name: nginx
        image: nginx:1.25-alpine
        ports:
        - containerPort: 80
EOF

kubectl apply -f nginx-deployment.yaml
kubectl get pods -l app=web-app
```

Explanation

The Deployment is named **web-app** and creates **2 replicas**. Both pods use the **nginx:1.25-alpine** image and expose container port 80.

Step 4.2: Create the NodePort Service

```
cat <<EOF > nginx-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: web-app-svc
spec:
  type: NodePort
  selector:
    app: web-app
  ports:
  - protocol: TCP
    port: 80
    targetPort: 80
    nodePort: 30080
EOF

kubectl apply -f nginx-service.yaml
kubectl get svc web-app-svc
```

The Service selects pods labelled **app: web-app** and exposes port 80 through fixed NodePort **30080**.

Step 4.3: Expose Minikube through the EC2 Public Interface

```
kubectl port-forward --address 0.0.0.0 svc/web-app-svc 30080:80 > /dev/null 2>&1 &
```

This binds port 30080 on all EC2 interfaces so traffic can reach the service from outside the EC2 instance.

Step 4.4: Verify the Application

```
# Test locally on EC2
curl http://localhost:30080

# Open from your laptop browser
http://<YOUR-EC2-PUBLIC-IP>:30080
```

7. PHASE 5 – CLEAN UP AWS RESOURCES

Clean up Kubernetes resources and Minikube when the exercise is complete to prevent unnecessary AWS charges.

```
# Delete Kubernetes Service
kubectl delete -f nginx-service.yaml

# Delete Kubernetes Deployment
kubectl delete -f nginx-deployment.yaml

# Stop and delete Minikube
minikube stop && minikube delete
```

Then terminate the EC2 instance from:

```
AWS Console → EC2 Dashboard → Instances → Select Instance → Instance State → Terminate
```

8. COMPLETE COMMAND SEQUENCE

```
# SSH
chmod 400 k8s-key.pem
ssh -i "k8s-key.pem" ubuntu@<EC2-PUBLIC-IP>

# Docker
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl wget apt-transport-https ca-certificates gnupg lsb-release
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu
sudo apt update && sudo apt install -y docker-ce docker-ce-cli containerd.io
sudo usermod -aG docker $USER
newgrp docker
docker ps

# kubectl
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
kubectl version --client

# Minikube
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
rm minikube-linux-amd64
minikube version

# Cluster
minikube start --driver=docker --cpus=2 --memory=3072
minikube status
kubectl get nodes
kubectl cluster-info

# Deploy
kubectl apply -f nginx-deployment.yaml
kubectl get pods -l app=web-app
kubectl apply -f nginx-service.yaml
kubectl get svc web-app-svc

# Expose
kubectl port-forward --address 0.0.0.0 svc/web-app-svc 30080:80 > /dev/null 2>&1 &

# Verify
curl http://localhost:30080
# Browser: http://<YOUR-EC2-PUBLIC-IP>:30080
```

9. SIMPLE EXPLANATION FOR VIVA

Q1. What is Kubernetes?
Kubernetes is a platform used to manage and run containerized applications.

Q2. What is Minikube?
Minikube runs a small Kubernetes cluster locally or on a lab server.

Q3. Why is t3.medium used?
The guide specifies it because Minikube needs at least 2 vCPUs and sufficient memory for the cluster.

Q4. What is kubectl?
kubectl is the command-line tool used to interact with a Kubernetes cluster.

Q5. What is a Deployment?
A Deployment manages the desired number of application pod replicas.

Q6. How many Nginx replicas are created?
Two replicas.

Q7. What image is used?
nginx:1.25-alpine.

Q8. What is a NodePort?
A Kubernetes Service type that exposes a service through a port on the node.

Q9. Which NodePort is used?
30080.

Q10. Why is port-forward used here?
The Minikube Docker network is isolated from the EC2 host, so port-forwarding binds the service to an EC2 interface.

Q11. How do you access the application?
Use http://:30080 in a browser.

Q12. How do you check pods?
kubectl get pods -l app=web-app.

Q13. How do you check the service?
kubectl get svc web-app-svc.

Q14. How do you remove the deployment?
kubectl delete -f nginx-deployment.yaml.

Q15. How do you remove Minikube?
minikube stop && minikube delete.

10. RESULT

The AWS EC2 Kubernetes lab was configured according to the exercise. Docker, kubectl and Minikube were installed; a Minikube cluster was started with the Docker driver; an Nginx Deployment with two replicas and a NodePort Service on port 30080 were created; the service was exposed through the EC2 interface and verified using curl/browser access. Finally, the Kubernetes resources, Minikube cluster and EC2 instance can be cleaned up.

— END OF SOLVED ASSIGNMENT —